Security Audit

Full Audit Report For MoonBull

Client: MoonBull
https://www.moonbull.io/

February 3, 2026

Version: 1.0

Document ID: 455BD3D8648AD95DF0A0FC44F595E7B5

Table of Contents

1 Disclaimer

This security assessment does not guarantee the security of the program instruction received from the client, herein referred to as “Source code.”

The Service Provider will not be held liable for any legal liability arising from errors in the security assessment. The responsibility for ensuring the security of the program instruction will be solely with the Client, herein referred to as “Service User.” The Service User agrees not to hold the Service Provider liable for any such liability.

By contract, the Service Provider is committed to conducting security assessments with integrity, professional ethics, and transparency in delivering security assessments to users. The Service Provider reserves the right to postpone the delivery of the security assessment if necessary, regardless of the reason for the delay. The Service Provider will not be held responsible for any delayed security assessments.

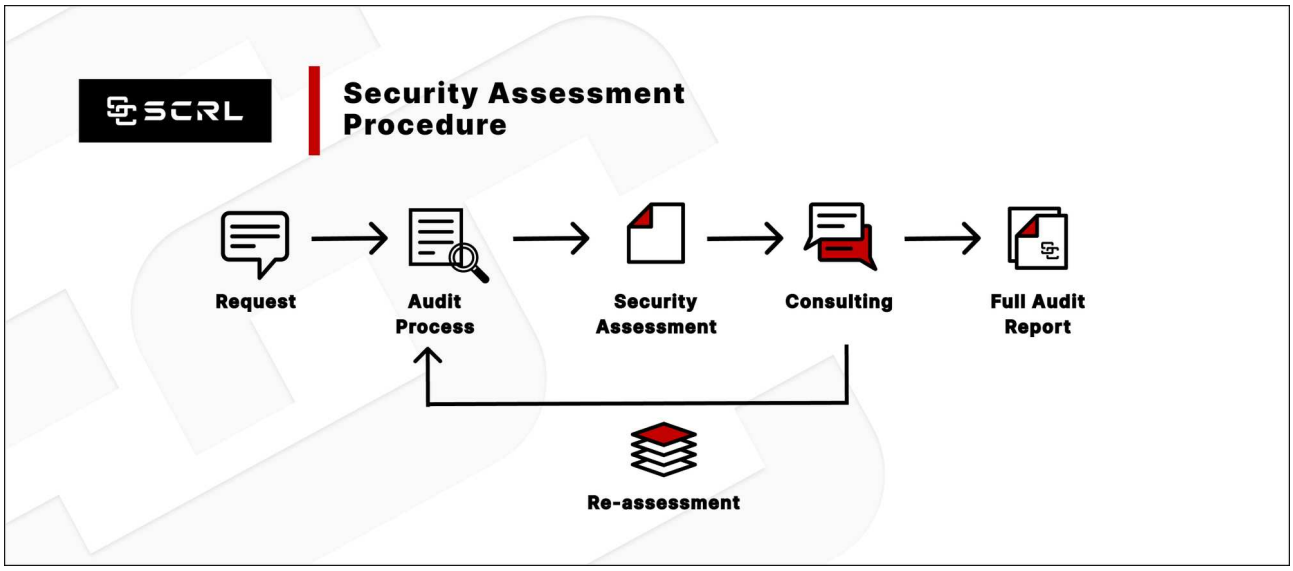
If the Service Provider identifies a vulnerability, we will notify the Service User via a Preliminary Report, which will be maintained in confidence for security purposes. The Service Provider disclaims responsibility in the event of any attacks occurring before or after conducting a security assessment. All responsibility for ensuring the security of the program instruction will be solely with the Service User.

Please be advised that the Security Assessment is not an investment advisory document and should not be interpreted as such. We hereby disclaim any liability arising from any investment:

- Security Assessment Is **Not Financial/Investment Advice** Any loss arising from any investment in any project is the responsibility of the investor.
- SCRL **disclaims** any liability incurred, whether it's **Rugpull, Abandonment, Soft Rugpull, Exploit, Exit Scam.**
- **Cryptocurrency and digital token involve high risks; investors may lose all investment money and should study information carefully and make investments according to own risk profile.**
- Please thoroughly comprehend any aspect of a project, including liquidity checks and pre-sale stages, and assume the associated risks independently. We hereby disclaim any liability for any investment decisions made by you.

For full Legal / TOS / Privacy Policy please visit: *https://scrl.io/legal*

2 Security Assessment Procedure



- **Request:** The client must submit a formal request and follow the procedure. By submitting the source code and agreeing to the terms of service.
- **Audit Process:** Check for vulnerabilities and vulnerabilities from source code obtained by experts using formal verification methods, including using powerful tools such as Static Analysis, SWC Registry, Dynamic Security Analysis, Automated Security Tools, CWE, Syntax & Parameter Check with AI ,WAS (Warning Avoidance System a python script tools powered by SCRL).
- **Security Assessment:** Deliver Preliminary Security Assessment to clients to acknowledge the risks and vulnerabilities.
- **Consulting:** Discuss on risks and vulnerabilities encountered by clients to apply to their source code to mitigate risks.
- **Re-assessment:** Reassess the security when the client implements the source code improvements and if the client is satisfied with the results of the audit. We will proceed to the next step.
- **Full Audit Report:** SCRL provides clients with official security assessment reports informing them of risks and vulnerabilities. Officially and it is assumed that the client has been informed of all the information.

3 Risk Rating

Risk rating using this commonly defined: **Risk rating = impact * confidence**

- **Impact:** The severity and potential impact of an attacker attack
- **Confidence:** Ensuring that attackers expose and use this vulnerability

Confidence & Impact [Likelihood]	Low	Medium	High
Low	**Informational**	**Low**	**Medium**
Medium	**Low**	**Medium**	**High**
High	**Medium**	**High**	**Critical**

Severity is a risk assessment It is calculated from the Impact and Confidence values using the following calculation methods, Risk rating = impact * confidence It is categorized into 7 categories severity based

Severity Risk	Description
Critical	Critical severity is assigned to security vulnerabilities that pose a severe threat to the smart contract and the entire blockchain ecosystem.
High	High-severity issues should be addressed quickly to reduce the risk of exploitation and protect users' funds and data.
Medium	It's essential to fix medium-severity issues in a reasonable timeframe to enhance the overall security of the smart contract.
Low	While low-severity issues can be less urgent, it's still advisable to address them to improve the overall security posture of the smart contract.
Informational	Used to categorize security findings that do not pose a direct security threat to the smart contract or its users. Instead, these findings provide additional information, recommendations

4 Category

Category	Description
Centralization	Centralization Risk is The risk incurred by a sole proprietor, such as the Owner being able to change something without permission
Economics Risk	Economics Risk is Risks that may affect the economic mechanism system, such as the ability to increase Mint token
Logical Issue	Logical Issue is that can cause errors to core processing, such as any prior operations that cause background processes to crash.
Authorization	Authorization is Possible pitfalls from weak coding allows unrelated people to take any action to modify the values.
Mathematical	Any erroneous arithmetic operations affect the operation of the system or lead to erroneous values.
Naming Conventions	naming variables that may affect code understanding or naming inconsistencies
Security Risk	Security Risk of loss or damage if it's no mitigate
Coding Style	Coding Style is Tips coding for efficiency performance
Best Practices	Best Practices is suggestions for improvement
Optimization	Optimization is performance improvement
Gas Optimization	Gas Optimization is increase performance to avoid expensive gas
Dead Code	Dead Code having unused code This may result in wasted resources and gas fees.
Input Validation	Proper input validation is essential to ensure that a smart contract processes only valid and anticipated data. Failure to implement robust input validation mechanisms may lead to various security vulnerabilities, including logic manipulation, unauthorized access, and unintended contract behavior.

5 Executive Summary

For this security assessment, SCRL received a request on **January 29, 2026**

Audit Score	Project Website	X (Twitter)	Telegram
9.8	https://www.moonbull.io/	https://x.com/MoonBullX	https://t.me/MoonBullCoin

Client	Confidential	Audit Method	Language
MoonBull	Public	Static Analysis + Dynamic Analysis + Manual Review + Formal Verification + Hyper Fuzzing + WAS (Custom Detector)	Solidity

Vulnerabilities Summary:

chart

2	1	0	1	0
Total Findings	**Resolved**	**Mitigated**	**Acknowledge**	**Declined**

0	Critical		Critical severity is assigned to security vulnerabilities that pose a severe threat to the smart contract and the entire blockchain ecosystem.
0	High		High-severity issues should be addressed quickly to reduce the risk of exploitation and protect users funds and data.
0	Medium		It essential to fix medium-severity issues in a reasonable timeframe to enhance the overall security of the smart contract.
1	Low	Acknowledge: 1	While low-severity issues can be less urgent, it still advisable to address them to improve the overall security posture of the smart contract.
1	Informational	Resolved: 1	Used to categorize security findings that do not pose a direct security threat to the smart contract or its users. Instead, these findings provide information

6 Audit Information

Audit Scope:

File	SHA1-Hash
src/Claim.sol	02e5cc9872176389e71f405925ddfd998fd114f2

Onchain Scope:

Chain	Contract Address
Not Deployed	-

Security Assessment Author:

Name	Role
Chinnakit J.	CEO & Founder
Ronny C.	CTO & Head of Security Researcher
Mark K.	Security Researcher

Smart Contract Audit Summary:

Digital Sign:

ID: 27173C85-F06A-4742-BF56-CDC643A385F5
Digitally signed by <contact@scrl.io>
February 03, 2026 03:03 PM +07

SWC Checklist

ID	Title	Scanning	Result
SWC-100	Function Default Visibility	Completed	No Risk
SWC-101	Integer Overflow and Underflow	Completed	No Risk
SWC-102	Outdated Compiler Version	Completed	No Risk
SWC-103	Floating Pragma	Completed	No Risk
SWC-104	Unchecked Call Return Value	Completed	No Risk
SWC-105	Unprotected Ether Withdrawal	Completed	No Risk
SWC-106	Unprotected SELFDESTRUCT Instruction	Completed	No Risk
SWC-107	Reentrancy	Completed	No Risk
SWC-108	State Variable Default Visibility	Completed	No Risk
SWC-109	Uninitialized Storage Pointer	Completed	No Risk
SWC-110	Assert Violation	Completed	No Risk
SWC-111	Use of Deprecated Solidity Functions	Completed	No Risk
SWC-112	Delegatecall to Untrusted Callee	Completed	No Risk
SWC-113	DoS with Failed Call	Completed	No Risk
SWC-114	Transaction Order Dependence	Completed	No Risk
SWC-115	Authorization through tx.origin	Completed	No Risk
SWC-116	Block values as a proxy for time	Completed	No Risk
SWC-117	Signature Malleability	Completed	No Risk
SWC-118	Incorrect Constructor Name	Completed	No Risk
SWC-119	Shadowing State Variables	Completed	No Risk
SWC-120	Weak Sources of Randomness from Chain Attributes	Completed	No Risk
SWC-121	Missing Protection against Signature Replay Attacks	Completed	No Risk

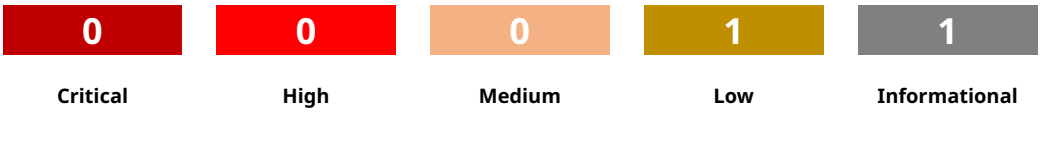
ID	Title	Scanning	Result
SWC-122	Lack of Proper Signature Verification	Completed	No Risk
SWC-123	Requirement Violation	Completed	No Risk
SWC-124	Write to Arbitrary Storage Location	Completed	No Risk
SWC-125	Incorrect Inheritance Order	Completed	No Risk
SWC-126	Insufficient Gas Griefing	Completed	No Risk
SWC-127	Arbitrary Jump with Function Type Variable	Completed	No Risk
SWC-128	DoS With Block Gas Limit	Completed	No Risk
SWC-129	Typographical Error	Completed	No Risk
SWC-130	Right-To-Left-Override control character (U+202E)	Completed	No Risk
SWC-131	Presence of unused variables	Completed	No Risk
SWC-132	Unexpected Ether balance	Completed	No Risk
SWC-133	Hash Collisions With Multiple Variable Length Arguments	Completed	No Risk
SWC-134	Message call with hardcoded gas amount	Completed	No Risk
SWC-135	Code With No Effects	Completed	No Risk
SWC-136	Unencrypted Private Data On-Chain	Completed	No Risk

OWASP Smart Contract Top 10 2025 Checklist

ID	Title	Scanning	Result
SC01:2025	Access Control Vulnerabilities	Completed	No Risk
SC02:2025	Price Oracle Manipulation	Completed	No Risk
SC03:2025	Logic Errors	Completed	No Risk
SC04:2025	Lack of Input Validation	Completed	No Risk
SC05:2025	Reentrancy Attacks	Completed	No Risk
SC06:2025	Unchecked External Calls	Completed	No Risk
SC07:2025	Flash Loan Attacks	Completed	No Risk
SC08:2025	Integer Overflow and Underflow	Completed	No Risk
SC09:2025	Insecure Randomness	Completed	No Risk
SC10:2025	Denial of Service (DoS) Attacks	Completed	No Risk

7 Findings List

chart



This security assessment report for **MoonBull** a total of **2 vulnerabilities.** The evaluation was conducted using the **Static Analysis + Dynamic Analysis + Manual Review + Formal Verification + Hyper Fuzzing + WAS (Custom Detector)** security assessment methodology.

ID	Vulnerabilities	Severity	Category	Status
CEN-01	Centralization Risk	**Low**	**Centralization**	**Acknowledge**
UCE-01	Use Custom Errors	**Informational**	**Gas Optimization**	**Resolved**

8 Findings

CEN-01: Centralization Risk

Vulnerabilities	Severity	Category	Status
Centralization Risk	Low	Centralization	Acknowledge

Finding Code

```
File: Claim.sol

8: contract Airdrop is Ownable {

24: ) Ownable(msg.sender) {

43: function withdrawUnclaimed() external onlyOwner {
```

Description

Centralization Risk

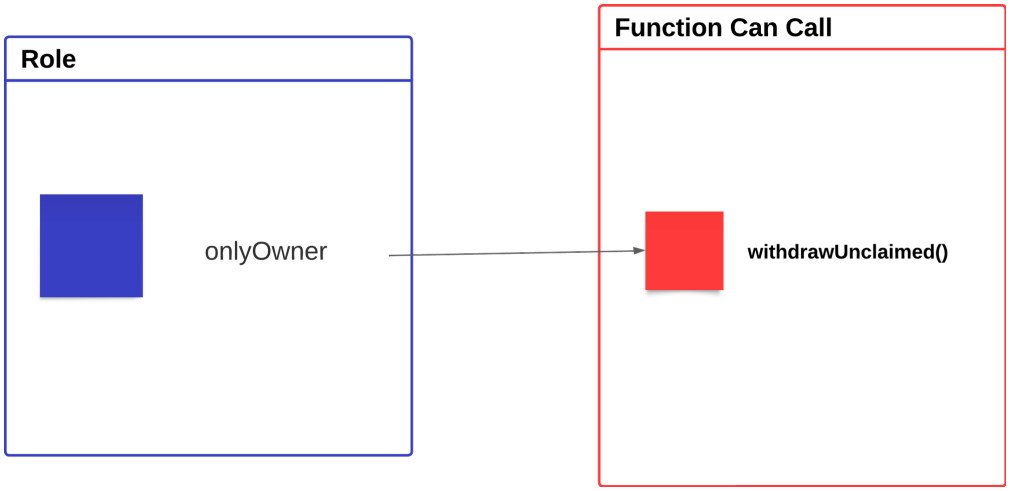


Figure 1 - This image shows who has privileges to call the function.

Recommendation

In terms of timeframes, there are three categories: short-term, long-term, and permanent.

For short-term solutions, a combination of timelock and multi-signature (2/3 or 3/5) can be used to mitigate risk by delaying sensitive operations and avoiding a single point of failure in key management. This includes implementing a timelock with a reasonable latency, such as 48 hours, for privileged operations; assigning privileged roles to multi-signature wallets to prevent private key compromise; and sharing the timelock contract and multi-signer addresses with the public via a medium/blog link.

For long-term solutions, a combination of timelock and DAO can be used to apply decentralization and transparency to the system. This includes implementing a timelock with a reasonable latency, such as 48 hours, for privileged operations; introducing a DAO/governance/voting module to increase transparency and user

involvement; and sharing the timelock contract, multi-signer addresses, and DAO information with the public via a medium/blog link.

Finally, permanent solutions should be implemented to ensure the ongoing security and protection of the system.

- Renounce the ownership and cannot turn back to claim a privileges to execution a function

OR

- Remove a contain centralization risk function and deployed a new contract

References

-

UCE-01: Use Custom Errors

Vulnerabilities	Severity	Category	Status
Use Custom Errors	Informational	Gas Optimization	Resolved

Finding Code

```
File: Claim.sol

31: require(block.timestamp >= claimStartTime, "claim not yet available");

32: require(!hasClaimed[msg.sender], "already claimed");

35: require(MerkleProof.verify(merkleProof, merkleRoot, node), "invalid proof");
```

Description

The contract uses `require()` statements with revert strings to handle errors and enforce conditions. While this is syntactically valid, it is not the most efficient method in terms of gas consumption.

Starting with **Solidity 0.8.4**, developers can define and use custom errors as a more gas-efficient and structured alternative to revert strings. Custom errors reduce deployment and runtime costs by avoiding the storage of long string literals in contract bytecode.

Recommendation

Refactor require statements with revert strings into custom errors, particularly in frequently used functions or large codebases. This approach improves gas efficiency and allows for more structured error handling, enhancing the overall quality and professionalism of the code.

Alleviation

Moonbull Team has already resolved this issue by use custom error

```
    // Custom errors
    error ClaimNotYetAvailable();
    error AlreadyClaimed();
    error InvalidProof();
```

and refactor

```
        if (block.timestamp < claimStartTime) revert ClaimNotYetAvailable();
        if (hasClaimed[msg.sender]) revert AlreadyClaimed();
        if (!MerkleProof.verify(merkleProof, merkleRoot, node)) revert InvalidProof();
```

References

Control Structures – Solidity Docs *https://docs.soliditylang.org/en/latest/control-structures.html#custom-errors*

A Appendix

A.1 Source Units in Scope

Source Units Analyzed: **1**
Source Units in Scope: **1** (**100%**)

Type	File	Logic Contracts	Interfaces	Lines	nLines	nSLOC	Comment Lines	Complex. Score	Capabilities
📝	src/Claim.sol	1	***	52	52	37	3	29	🧮
📝	**Totals**	**1**	***	**52**	**52**	**37**	**3**	**29**	🧮

Legend: [➖]

- **Lines**: total lines of the source unit
- **nLines**: normalized lines of the source unit (e.g. normalizes functions spanning multiple lines)
- **nSLOC**: normalized source lines of code (only source-code lines; no comments, no blank lines)
- **Comment Lines**: lines containing single or block comments
- **Complexity Score**: a custom complexity score derived from code statements that are known to introduce code complexity (branches, loops, calls, external interfaces, ...)

A.2 Visibility, Mutability, Modifier function testing

Components

📝****Contracts	📚****Libraries	🔍****Interfaces	🎨****Abstract
1	0	0	0

Exposed Functions

This section lists functions that are explicitly declared public or payable. Please note that getter methods for public stateVars are not included.

🌐****Public	💰****Payable
2	0

External	Internal	Private	Pure	View
2	3	0	0	0

StateVariables

Total	🌐****Public
4	4

Capabilities

Solidity Versions observed	🧪** Experimental Features**	💰** Can Receive Funds**	📱** Uses Assembly**	💣** Has Destroyable Contracts**
^0.8.9		***	***	***

📤** Transfers ETH**	⚡**** Low-Level Calls	👥** DelegateCall**	🧮** Uses Hash Functions**	🔖** ECRecover**	🌀** New/Create/Create2**
***	***	***	yes	***	***

♻️** TryCatch**	Σ Unchecked
***	***

Dependencies / External Imports

Dependency / Import Path	Count
@openzeppelin/contracts/access/Ownable.sol	1

Dependency / Import Path	Count
@openzeppelin/contracts/token/ERC20/utils/SafeERC20.sol	1
@openzeppelin/contracts/utils/cryptography/MerkleProof.sol	1

A.3 Contracts Description Table

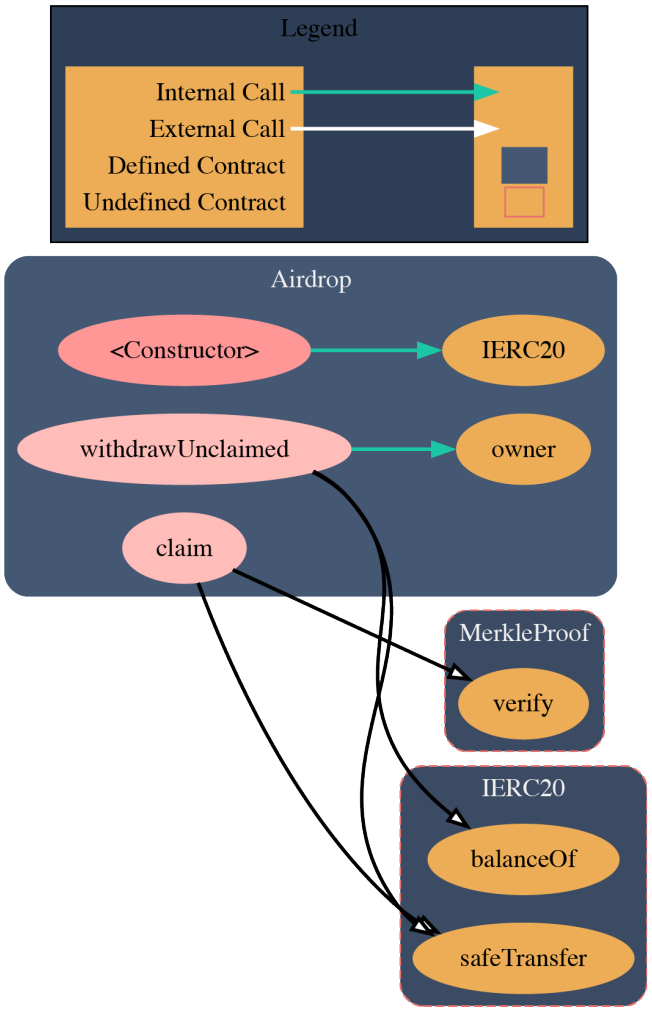
Contracts Description Table

Contract	Type	Bases		
└	**Function Name**	**Visibility**	**Mutability**	**Modifiers**
Airdrop	Implementation	Ownable		
└		Public ❗	🛑	Ownable
└	claim	External ❗	🛑	NO ❗
└	withdrawUnclaimed	External ❗	🛑	onlyOwner

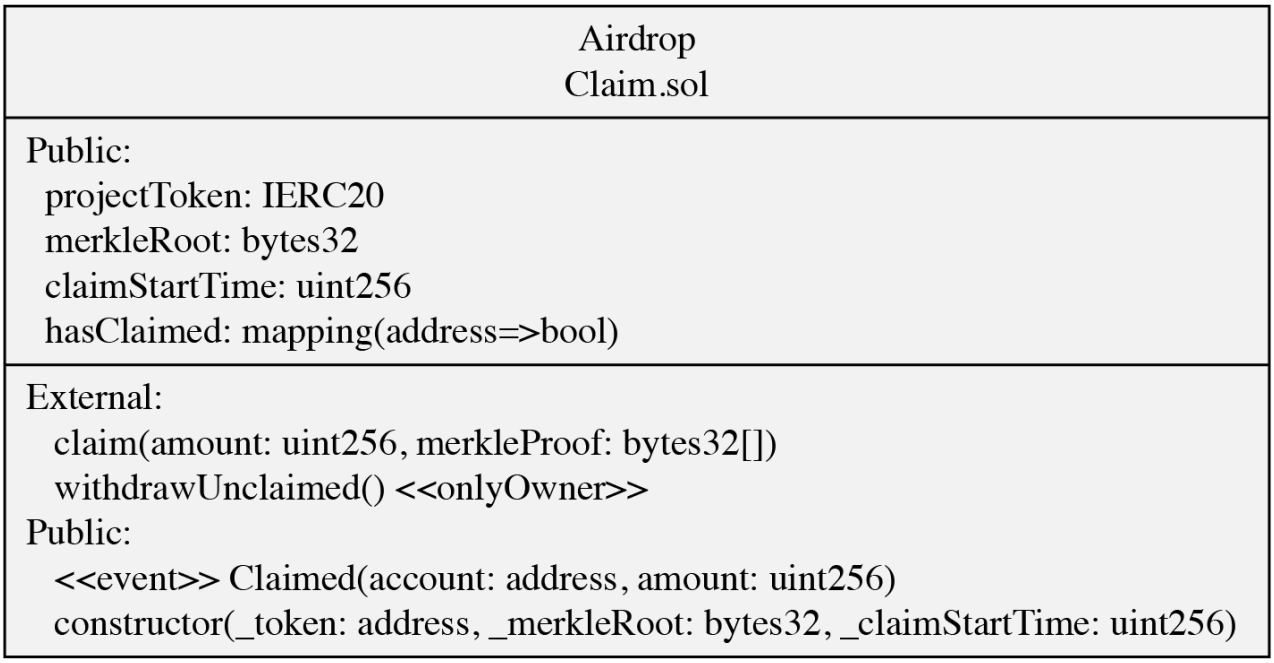
Legend

Symbol	Meaning
🛑	Function can modify state
💵	Function is payable

A.4 Call Graph



A.5 UML Class Diagram



A.6 Code Snippet

```
// SPDX-License-Identifier: UNLICENSED
pragma solidity ^0.8.9;

import "@openzeppelin/contracts/access/Ownable.sol";
import "@openzeppelin/contracts/token/ERC20/utils/SafeERC20.sol";
import "@openzeppelin/contracts/utils/cryptography/MerkleProof.sol";

contract Airdrop is Ownable {
    using SafeERC20 for IERC20;

    IERC20 public immutable projectToken;
    bytes32 public immutable merkleRoot;
    uint256 public immutable claimStartTime;

    // Track claimed status
    mapping(address => bool) public hasClaimed;

    // Custom errors
    error ClaimNotYetAvailable();
    error AlreadyClaimed();
    error InvalidProof();

    event Claimed(address indexed account, uint256 amount);

    constructor(
        address _token,
        bytes32 _merkleRoot,
        uint256 _claimStartTime
    ) Ownable(msg.sender) {
        projectToken = IERC20(_token);
        merkleRoot = _merkleRoot;
        claimStartTime = _claimStartTime;
    }

    function claim(uint256 amount, bytes32[] calldata merkleProof) external {
        if (block.timestamp < claimStartTime) revert ClaimNotYetAvailable();
        if (hasClaimed[msg.sender]) revert AlreadyClaimed();

        bytes32 node = keccak256(abi.encodePacked(msg.sender, amount));
        if (!MerkleProof.verify(merkleProof, merkleRoot, node)) revert InvalidProof();

        hasClaimed[msg.sender] = true;
        projectToken.safeTransfer(msg.sender, amount);

        emit Claimed(msg.sender, amount);
    }

    function withdrawUnclaimed() external onlyOwner {
        uint256 balance = projectToken.balanceOf(address(this));
        projectToken.safeTransfer(owner(), balance);
    }
}
```

About SCRL

SCRL is a dynamic cybersecurity team that provides security assessments, penetration testing, and incident investigations. We utilize specialized tools, expertise, and frameworks to enhance security measures. Additionally, we develop robust internal tools tailored to our organization's needs.
SCRL is committed to driving security for the blockchain industry and businesses alike.

Follow Us On:

Platform	Link
Website	*https://scrl.io/*
Twitter	*https://twitter.com/scrl_io*
Telegram	*https://t.me/scrl_io*
Medium	*https://scrl.medium.com/*